Numerical Precision Impact on AI Inference Across CPU, GPU, TPU and NPU

Computer Architecture — Individual Research Project

Submitted by:
Kashaf Sarfraz
BS Computer Science
FAST National University of Computer and Emerging Sciences
Faisalabad Campus

Submitted to:
Ms. Sumaira Mustafa

Academic Session: 2024–2028

Abstract

Artificial Intelligence (AI) applications increasingly depend on computationally intensive operations, particularly matrix multiplication and neural network inference. The performance of these operations is influenced not only by the processing hardware but also by the numerical precision used during computation. Different processor architectures, including Central Processing Units (CPUs), Graphics Processing Units (GPUs), Tensor Processing Units (TPUs), and Neural Processing Units (NPUs), are designed to handle computational workloads in different ways.

This research investigates how numerical precision affects AI performance across different computing architectures. The experimental comparison focuses on an Intel Xeon CPU, an NVIDIA Tesla T4 GPU, and a Google Cloud TPU v5e environment. The study examines matrix multiplication, memory bandwidth, numerical precision, host-device data transfer, and selected AI workloads. It also applies computer architecture concepts such as the Roofline Model, Amdahl's Law, CPU performance analysis, and thread scaling.

The experiments were conducted using Python and PyTorch in Google Colab. The collected results show noticeable performance differences between the tested devices. For a 1024 × 1024 matrix multiplication experiment, the CPU recorded approximately 133.08 GFLOPS, the NVIDIA T4 GPU achieved approximately 2941.3 GFLOPS, and the TPU v5e recorded approximately 5577.9 GFLOPS. The results also demonstrate that reduced precision does not provide the same performance improvement on every architecture. For example, FP16 performed substantially faster than FP32 on the tested NVIDIA T4 GPU, while reduced-precision operations were considerably slower in the tested CPU software environment.

The research highlights that selecting an appropriate numerical format requires consideration of hardware capabilities, software support, computational workload, memory movement, and numerical accuracy. The findings provide an experimental understanding of how computer architecture influences AI inference performance.

Keywords: Computer Architecture, Artificial Intelligence, Numerical Precision, CPU, GPU, TPU, NPU, FP32, FP16, BF16, AI Inference, Roofline Model.

1. Introduction

Artificial Intelligence has become an important part of modern computing. Applications such as image recognition, natural language processing, recommendation systems, and intelligent automation rely on machine learning models that perform large numbers of mathematical operations. As these models become more complex, their computational requirements also increase.

A major part of AI computation involves matrix multiplication and other numerical operations. These operations are performed repeatedly during neural network training and inference. Their execution time depends on several factors, including processor architecture, memory bandwidth, parallel processing capabilities, software optimization, and numerical precision.

Numerical precision describes the number of bits used to represent a value during computation. Common formats include FP32 (32-bit floating point), FP16 (16-bit floating point), BF16 (16-bit brain floating point), and INT8 (8-bit integer). Higher-precision formats generally provide a wider or more detailed representation of numerical values, while lower-precision formats can reduce storage requirements and computational cost when supported efficiently by the hardware.

Modern computing devices are designed with different approaches to processing these operations. A Central Processing Unit (CPU) is a general-purpose processor designed to handle a wide range of tasks. A Graphics Processing Unit (GPU) contains many processing units that can execute suitable operations in parallel. A Tensor Processing Unit (TPU) is designed to accelerate machine learning computations, particularly tensor operations. A Neural Processing Unit (NPU) is another specialized processor category developed to perform AI-related operations efficiently.

Although these devices can execute AI workloads, their performance is not identical. A numerical format that provides good performance on one device may not provide the same benefit on another. Software frameworks and hardware support also influence the final result.

This research therefore examines the relationship between numerical precision and AI performance across different processor architectures. The main focus is to understand how precision formats affect computational speed, memory usage, data movement, and the execution of selected AI workloads.

1.1 Background of the Study

Traditional computing systems commonly use FP32 for numerical calculations because it provides a useful balance between precision and computational range. However, modern AI workloads often require a large number of calculations, making computational efficiency increasingly important.

Lower-precision formats such as FP16 and BF16 have become widely used in AI computing. These formats can reduce the amount of data transferred and stored. They may also allow compatible processors to perform more operations in less time. INT8 can provide further reductions in data size and can be useful in inference workloads when the model and hardware support quantized operations.

However, lower precision also introduces numerical differences. Rounding errors and limited representation range may affect the output of mathematical operations and neural networks. Therefore, performance improvements must be considered alongside numerical accuracy.

The architecture of the processor is another important factor. CPUs, GPUs, and TPUs have different internal designs and execution mechanisms. Their ability to benefit from a particular numerical format depends on the available hardware units, memory system, instruction support, and software implementation.

Understanding these differences is useful when selecting computing hardware for AI applications. It also provides a practical way to study important computer architecture concepts through real experimental measurements.

1.2 Problem Statement

AI workloads require significant computational resources, and their performance can vary considerably across different processor architectures. Although reduced-precision formats are often used to improve AI performance, they do not necessarily provide the same benefits on every device.

The main problem addressed in this research is the lack of a direct experimental understanding of how numerical precision influences AI computation across different architectures. A processor may support a particular precision format but still perform poorly with it because of hardware limitations, software implementation, or workload characteristics.

Furthermore, computational performance cannot be evaluated through processing speed alone. Memory bandwidth, host-device transfer overhead, numerical error, and the type of AI workload can also influence the overall result.

This study addresses these issues by comparing selected CPU, GPU, and TPU environments through practical experiments. It also discusses NPU architecture as part of the broader comparison, while clearly distinguishing conceptual discussion from hardware measurements.

1.3 Research Objectives

The main objectives of this research are:

1. To study the architectural differences between CPUs, GPUs, TPUs, and NPUs in relation to AI computation.
2. To investigate the effect of numerical precision formats, particularly FP32, FP16, and BF16, on computational performance.
3. To measure and compare matrix multiplication performance across the tested CPU, GPU, and TPU environments.
4. To examine memory bandwidth and its influence on computational performance.
5. To investigate host-device data transfer overhead in the tested GPU environment.
6. To evaluate the performance of selected AI workloads, including CNN-based and Transformer-based models, where measurements are available.
7. To study the relationship between computational performance, memory bandwidth, and arithmetic intensity using the Roofline Model.
8. To apply Amdahl's Law and CPU performance analysis to understand the impact of parallel processing.
9. To examine the relationship between numerical precision and computational accuracy.
10. To identify the advantages and limitations of different processor architectures for AI workloads.

1.4 Research Questions

This research is guided by the following questions:

1. How does numerical precision affect the performance of AI computations on CPU, GPU, and TPU architectures?
2. Which tested architecture provides better matrix multiplication performance under the recorded experimental conditions?
3. Does reduced precision always improve computational performance?
4. How do memory bandwidth and data transfer overhead influence AI workload execution?
5. What relationship exists between computational speed and numerical accuracy when using lower-precision formats?
6. How can architectural models such as the Roofline Model and Amdahl's Law help explain the measured performance differences?

1.5 Scope of the Research

This research focuses on the experimental evaluation of numerical precision and AI performance using three available computing environments:

- Intel Xeon CPU available through Google Colab.
- NVIDIA Tesla T4 GPU.
- Google Cloud TPU v5e environment.

The experimental work includes matrix multiplication, memory bandwidth measurements, precision comparisons, GPU host-device transfers, and selected AI workload experiments. A CNN training experiment is also used to compare training time and test accuracy between the available CPU and GPU environments.

The research discusses NPU architecture to provide a broader understanding of specialized AI processors. However, an NPU was not available for direct experimentation in the project environment. Therefore, no measured NPU performance values are claimed.

Similarly, experiments that could not be completed on every platform are treated as limitations rather than being presented as completed comparisons. The results are interpreted according to the actual devices, software versions, and experimental conditions used.

1.6 Significance of the Research

This research is significant because it connects theoretical computer architecture concepts with practical AI experiments. Instead of considering processor performance only through advertised specifications, the study uses measured results to understand how different devices behave during selected computational tasks.

The comparison helps explain why hardware support and numerical precision are important when running AI models. It also demonstrates that memory bandwidth, data movement, and software implementation can influence performance alongside the processor's computational capability.

The findings can help students and researchers develop a better understanding of hardware selection for AI workloads. In addition, the study provides practical experience with benchmarking, numerical error analysis, profiling, and performance models.

2. Computer Architecture Background

Computer architecture describes the internal organization and operation of a computing system. It explains how processors execute instructions, access memory, perform calculations, and communicate with other components.

For AI applications, processor architecture is particularly important because neural networks perform large numbers of repeated mathematical operations. These operations can require substantial processing power and memory bandwidth.

This section introduces the main processor architectures considered in the research and explains the numerical formats used in the experiments.

2.1 Central Processing Unit (CPU)

A Central Processing Unit is a general-purpose processor responsible for executing instructions and controlling many operations within a computer system. It is designed to handle different types of workloads, including operating system tasks, application logic, data processing, and mathematical calculations.

A CPU generally contains a limited number of powerful processing cores compared with a GPU. Each core can execute complex instructions and is supported by cache memory that helps reduce access time to frequently used data.

For AI workloads, CPUs can perform matrix operations and execute neural networks. However, highly parallel computations may take longer on a CPU than on a specialized accelerator, depending on the workload and implementation.

In this research, the CPU environment provided by Google Colab was used for matrix multiplication, memory bandwidth measurements, precision experiments, selected AI inference tasks, and a CNN training comparison.

Recorded CPU environment:

- Processor: Intel Xeon CPU @ 2.00 GHz
- Logical processors: 2
- L1 data cache: 32 KiB
- L2 cache: 1 MiB
- L3 cache: 38.5 MiB
- Framework: PyTorch 2.11.0+cu130

The exact Xeon processor model was not established in the recorded environment. Therefore, the CPU is identified using the available runtime information.

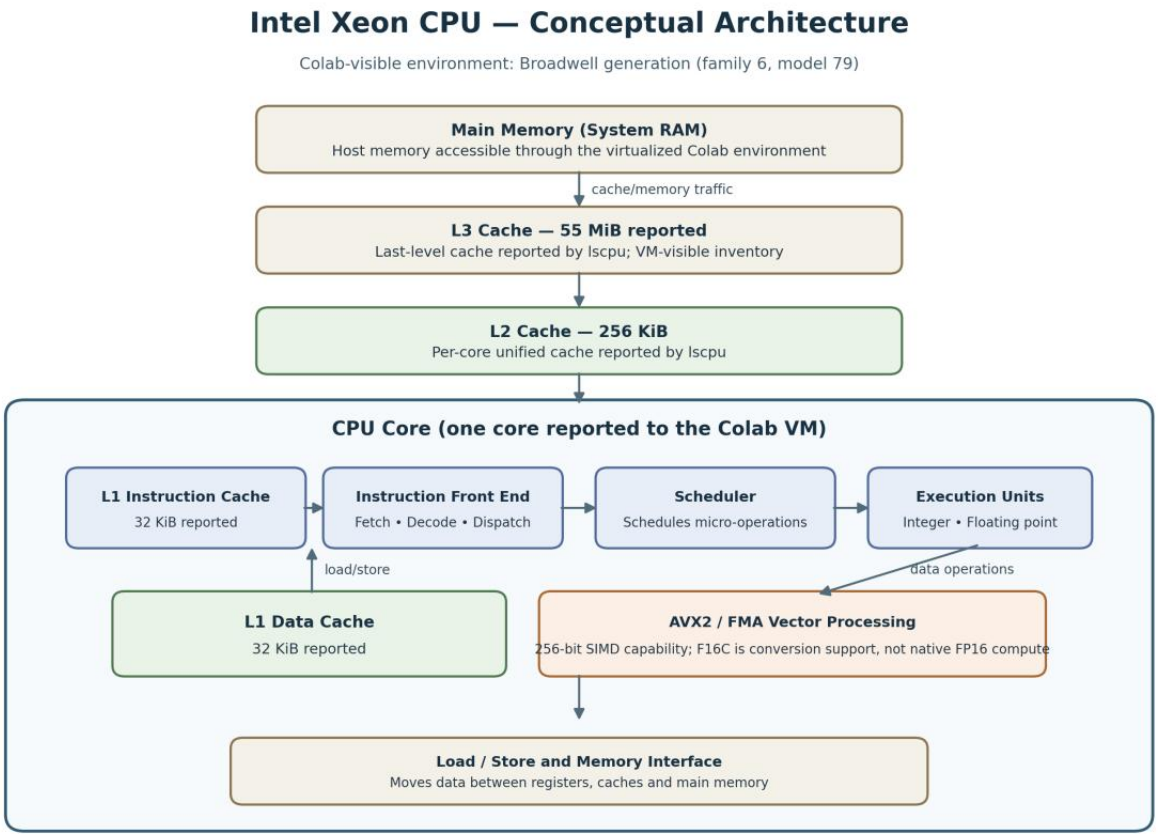


Figure 1: General Architecture of a Central Processing Unit (CPU).

2.2 Graphics Processing Unit (GPU)

A Graphics Processing Unit is designed to execute many operations in parallel. Although GPUs were originally developed for graphics processing, their parallel architecture makes them suitable for scientific computing and AI workloads.

Unlike a CPU, which generally uses fewer complex cores, a GPU contains a large number of smaller processing units. These units can perform similar operations on different data elements at the same time.

Modern GPUs also include specialized hardware for accelerating matrix operations and lower-precision calculations. This makes them particularly useful for neural network training and inference.

The NVIDIA Tesla T4 GPU was used in this research. It supports general-purpose GPU computation through CUDA and provides hardware acceleration for supported AI operations.

Recorded GPU environment:

- GPU: NVIDIA Tesla T4

- Reported memory: approximately 15,360 MiB
- CUDA version: 13.0
- Driver version: 580.82.07
- Framework: PyTorch 2.11.0+cu130
- CUDA availability: Confirmed

The GPU was used for matrix multiplication, memory bandwidth measurement, precision comparison, host-device transfer experiments, selected inference workloads, and CNN training.

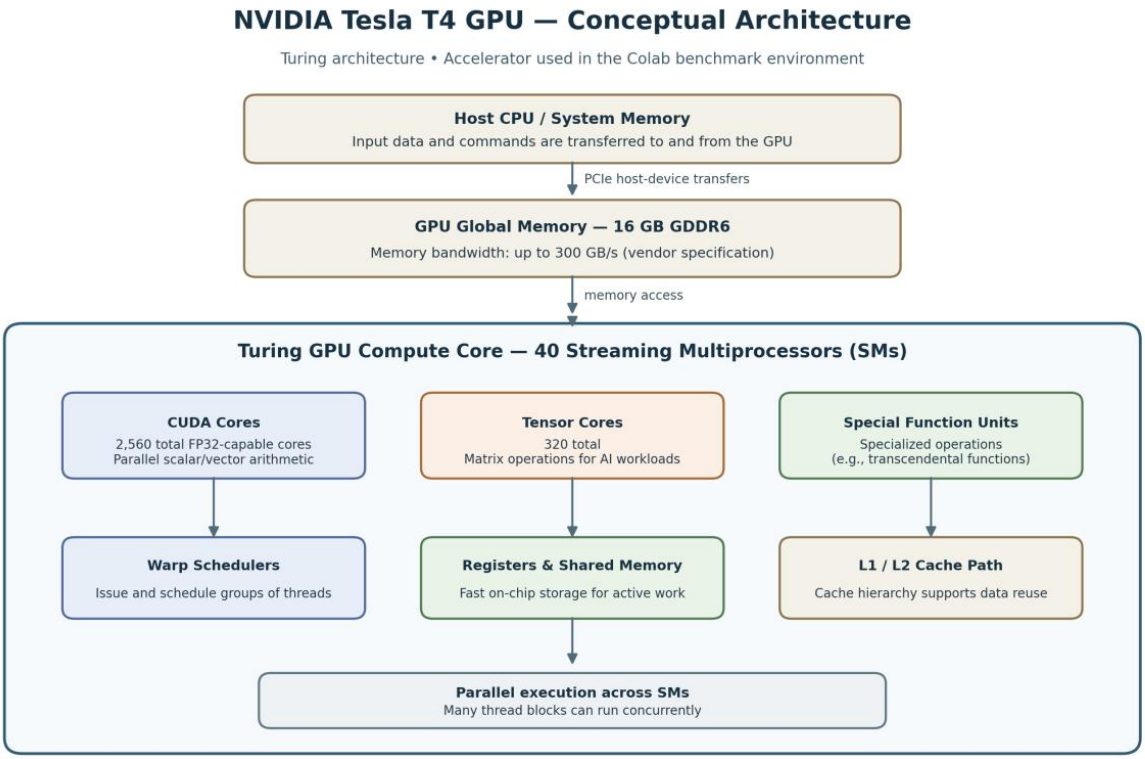


Figure 2: General Architecture of a Graphics Processing Unit (GPU).

2.3 Tensor Processing Unit (TPU)

A Tensor Processing Unit is a specialized processor developed by Google to accelerate machine learning operations. TPUs are designed to handle large-scale tensor calculations efficiently, particularly the matrix operations commonly used in neural networks.

Unlike general-purpose CPUs, TPUs focus on accelerating workloads that can be expressed as tensor operations. Their architecture is designed to provide high computational throughput for supported numerical formats and workloads.

TPUs are commonly used in machine learning tasks such as neural network training and inference. Their performance depends on the model, numerical format, software framework, and data movement requirements.

In this research, a Google Cloud TPU v5e environment was used for selected matrix multiplication, memory bandwidth, and precision experiments.

Recorded TPU environment:

- TPU: Google Cloud TPU v5e-1
- Framework environment: PyTorch 2.14.1+cpu with TPU integration
- Runtime: Google Colab TPU environment

Some TPU experiments were affected by runtime and usage limitations. Consequently, TPU results are reported only for experiments with recorded measurements.

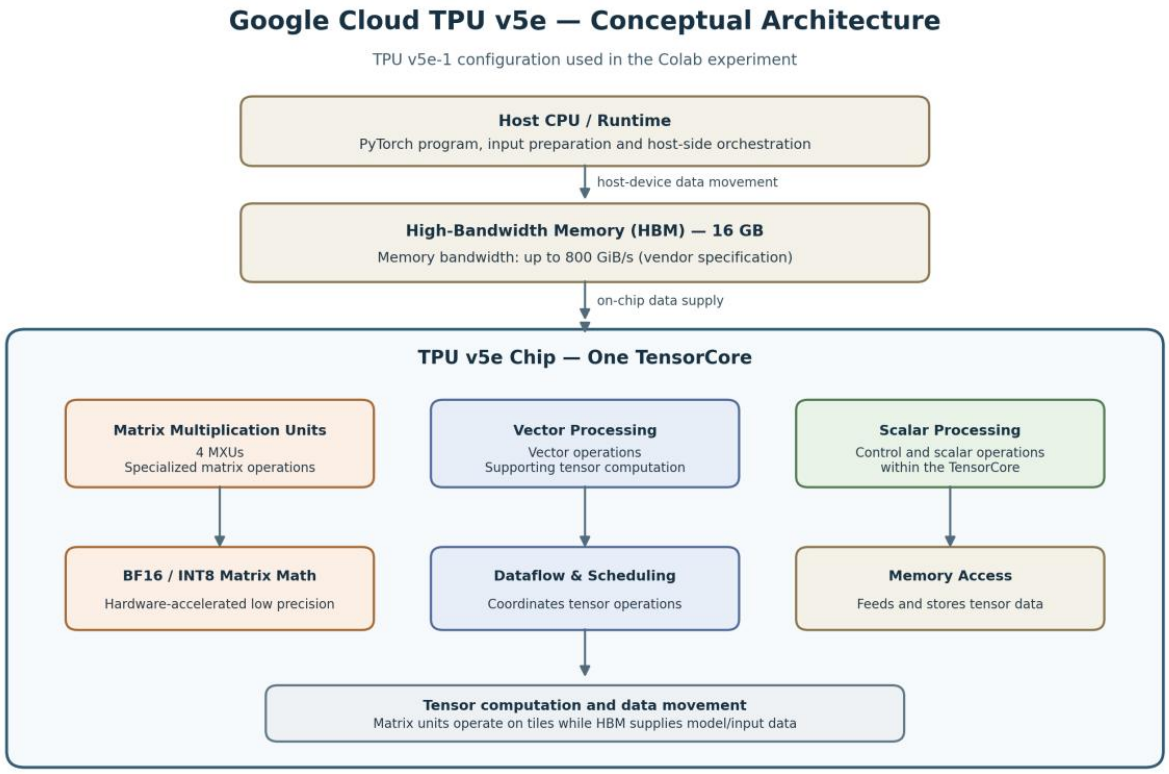


Figure 3: Simplified Architecture of a Tensor Processing Unit (TPU).

2.4 Neural Processing Unit (NPU)

A Neural Processing Unit is a specialized processor designed to accelerate artificial intelligence operations. NPUs are commonly integrated into mobile devices, personal computers, and other systems that require efficient local AI processing.

NPUs are designed to perform neural network operations with attention to power consumption, processing efficiency, and memory movement. They may support operations such as convolution, matrix multiplication, and quantized neural network inference.

An NPU differs from a CPU because it is optimized for AI-related workloads rather than general-purpose instruction execution. It also differs from a GPU in its architecture, target workloads, and integration into computing systems.

Although NPU architecture is discussed in this research, no physical NPU was available in the experimental environment. Therefore, the NPU comparison is conceptual and is not included in the measured performance results.

2.5 Comparison of Processor Architectures

The four processor categories differ in their design goals and typical applications.

Architecture	Main Purpose	Important Characteristics	Common AI Applications
CPU	General-purpose computing	Flexible instruction execution, cache hierarchy, multiple cores	Data preparation, general inference, application control
GPU	Parallel computing	Large number of parallel processing units, high memory bandwidth	Deep learning training and inference
TPU	Machine learning acceleration	Specialized tensor processing architecture	Neural network training and inference
NPU	Efficient AI processing	Specialized AI operations, often designed for power efficiency	On-device AI and neural network inference

The comparison shows that no single architecture is automatically the best choice for every AI workload. Performance depends on the computational requirements, precision format, memory system, and software support.

2.6 Numerical Precision Formats

Numerical precision determines how numerical values are represented and stored in a computing system. In AI applications, precision affects memory requirements, computational performance, and numerical accuracy.

The main formats considered in this research are FP32, FP16, and BF16.

2.6.1 FP32 (32-bit Floating Point)

FP32 uses 32 bits to represent a floating-point value. It is commonly used in scientific computing and traditional deep learning operations because it provides a relatively high level of numerical precision.

FP32 requires more memory than 16-bit formats. It may also require more computational resources on hardware that can accelerate lower-precision operations more efficiently.

2.6.2 FP16 (16-bit Floating Point)

FP16 uses 16 bits to represent a floating-point value. It reduces storage requirements compared with FP32 and can improve computational performance on hardware with suitable FP16 support.

However, FP16 has a smaller numerical range and less precision than FP32. As a result, rounding and representation errors may become more noticeable in some computations.

2.6.3 BF16 (Brain Floating Point)

BF16 is another 16-bit floating-point format commonly used in AI workloads. It uses fewer fraction bits than FP16 but retains a wider exponent range similar to FP32.

This characteristic makes BF16 useful for certain deep learning operations, where maintaining a suitable numerical range is important.

2.6.4 INT8 (8-bit Integer)

INT8 represents values using 8-bit integers. It is commonly used in quantized AI inference to reduce model size and computational requirements.

However, converting a model to INT8 generally requires an appropriate quantization process. The resulting accuracy depends on the model, calibration data, and quantization method.

INT8 was included in the research scope for architectural discussion, but no INT8 experimental result is claimed unless supported by a recorded benchmark.

2.7 Summary of the Background

CPU, GPU, TPU, and NPU architectures provide different approaches to processing AI workloads. CPUs offer flexibility, GPUs provide parallel processing capabilities, TPUs are specialized for tensor operations, and NPUs target efficient AI processing.

Similarly, numerical formats provide different balances between storage requirements, computational performance, and accuracy. FP32, FP16, BF16, and INT8 each have specific characteristics that influence their suitability for different applications.

The following sections present the theoretical performance concepts, experimental methodology, and measured results used to investigate these differences.

3. Theoretical Concepts and Performance Models

The performance of an AI application depends on several architectural factors. Processing speed is important, but memory bandwidth, numerical precision, data transfer, and parallel execution also affect the final performance.

This research uses mathematical performance measures and computer architecture models to interpret the experimental results.

3.1 Matrix Multiplication Performance

Matrix multiplication is one of the most common operations in artificial intelligence. Neural networks use matrix operations to perform transformations on input data, calculate weights, and generate outputs.

For two square matrices of dimension N × N, the approximate number of floating-point operations required for multiplication is:

[
F = 2N^3
]

Where:

- F = approximate number of floating-point operations.
- N = dimension of the square matrices.
- The factor 2 represents one multiplication and one addition for each multiply-accumulate operation.

The performance of matrix multiplication can be expressed in GFLOPS (Giga Floating Point Operations Per Second).

- $$GFLOPS = \frac{2 \times N^3}{T \times 10^9}$$
- T = execution time in seconds.
- (10^9) = number of floating-point operations represented by one giga-operation.

A higher GFLOPS value indicates that the device completed more floating-point operations per second under the measured conditions.

3.2 Memory Bandwidth

Memory bandwidth describes the rate at which data can be transferred between memory and a processor. It is commonly expressed in GB/s.

Memory bandwidth is important for AI workloads because models frequently read and write large amounts of data. Even when a processor has high computational capability, insufficient data delivery can limit its performance.

The measured memory bandwidth can be calculated using:

$$BW = \frac{D}{T}$$

Where:

- BW = memory bandwidth in bytes per second.
- D = amount of data transferred in bytes.
- T = time required for the transfer in seconds.

For reporting bandwidth in GB/s, the result is converted using the same unit convention as the benchmark implementation.

3.3 Numerical Precision and Storage Requirements

Different numerical formats use different numbers of bits to represent each value.

Format	Bits per Value	Storage per Value
FP32	32	4 bytes
FP16	16	2 bytes
BF16	16	2 bytes
INT8	8	1 byte

For the same number of values, FP16 and BF16 require approximately half the storage of FP32. INT8 requires approximately one-quarter of the storage of FP32.

However, lower storage requirements do not automatically guarantee faster execution. The processor must support efficient operations in the selected format, and the software framework must use that support correctly.

3.4 Numerical Error

Reduced-precision calculations can produce results that differ from FP32 calculations because of rounding and limited numerical representation.

For a reference result (x_i) and an approximate result (\hat{x}_i), the absolute error is $E_i = |x_i - \hat{x}_i|$

The maximum absolute error is:

$$E_{\max} = \max_i |x_i - \hat{x}_i|$$

The mean absolute error is:

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |x_i - \hat{x}_i|$$

The root mean square error is:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2}$$ Where:

- (x_i) = reference value.
- ({x}_i) = value calculated using the tested precision.
- n = total number of values.

These measurements help determine how much the calculated results differ from the reference output.

3.5 Host-Device Transfer Performance

When a GPU performs a computation, data may need to move between the CPU's memory and GPU memory. This movement introduces additional time beyond the computation itself.

The effective transfer bandwidth can be calculated as:

$$BW_{\text{transfer}} = \frac{D}{T_{\text{transfer}}}$$ Where:

- D = amount of transferred data.
- (T_{{transfer}}) = transfer time.

For smaller workloads, transfer overhead can represent a significant part of total execution time. Therefore, a device with high computational throughput may not always provide the lowest end-to-end latency.

3.6 Roofline Model

The Roofline Model is a performance analysis method that relates computational performance to arithmetic intensity and memory bandwidth.

Arithmetic intensity describes the number of operations performed for each byte of data transferred.

$$AI = \frac{\text{Floating Point Operations}}{\text{Bytes Transferred}}$$

The theoretical performance limit can be represented as:

$$P = \min(P_{\text{peak}}, BW_{\text{peak}} \times AI)$$Where:

- P = performance limit.
- (P_{{peak}}) = peak computational performance.
- (BW_{{peak}}) = memory bandwidth ceiling.
- AI = arithmetic intensity.

The model contains two main regions:

1. **Memory-bound region:** Performance is limited by the rate at which data can be supplied.
2. **Compute-bound region:** Performance is limited by the processing capability of the device.

A workload with low arithmetic intensity is more likely to be affected by memory bandwidth. A workload with high arithmetic intensity may benefit more from additional computational resources.

3.7 Amdahl's Law

Amdahl's Law explains the maximum speedup that can be achieved when only part of a program is parallelized.

It is expressed as:

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$ Where:

- S = theoretical speedup.
- P = fraction of the program that can be parallelized.
- N = number of processing units.
- (1-P) = serial fraction of the program.

The equation shows that the serial portion limits the overall speedup. Adding more processing units cannot completely remove the time required by operations that must execute sequentially.

3.8 CPU Performance and Iron Law

The CPU performance relationship commonly known as the Iron Law can be expressed as:

$$CPU\ Time = IC \times CPI \times Clock\ Cycle\ Time$$ Where:

- IC = instruction count.
- CPI = average clock cycles per instruction.
- Clock Cycle Time = time required for one clock cycle.

This relationship shows that CPU execution time depends on the number of instructions, the average number of cycles required per instruction, and the processor's clock cycle time.

Hardware performance counters can provide detailed information about these factors. However, such counters were not available through the project environment. Therefore, the CPU analysis in this research uses the available PyTorch profiler measurements rather than claiming complete hardware-counter profiling.

4. Experimental Methodology

This section describes the computing environments, software tools, workloads, and measurement approach used in the research.

4.1 Experimental Environment

The experiments were conducted using Google Colab with Google Drive for storing project files and results.

The recorded computing environments were:

Component	CPU	GPU	TPU
Processor	Intel Xeon @ 2.00 GHz	NVIDIA Tesla T4	Google Cloud TPU v5e-1
Available processing resources	2 logical processors	GPU parallel processing units	TPU tensor processing architecture
Framework environment	PyTorch 2.11.0+cu130	PyTorch 2.11.0+cu130	PyTorch 2.14.1+cpu with TPU integration
Main use	Matrix and AI workloads	Matrix and AI workloads	Selected matrix and precision experiments

The exact CPU model was not available in the runtime information. The CPU is therefore identified using the model description reported by Google Colab.

4.2 Software and Tools

The main software tools used in the project were:

- Python for experiment implementation.
- PyTorch for tensor operations and AI workloads.
- Google Colab for running experiments.
- Google Drive for storing CSV files, graphs, notebooks, and documentation.
- Matplotlib for plotting experimental results.
- Pandas for organizing and saving benchmark data.

The GPU environment reported CUDA 13.0 and NVIDIA driver version 580.82.07.

4.3 Matrix Multiplication Experiments

Matrix multiplication was used as a basic computational benchmark because it is an important operation in neural networks.

The experiments measured execution time and computational throughput on the available CPU, GPU, and TPU environments.

The main performance measurements included:

- Execution time.
- Matrix dimension.
- Computational throughput in GFLOPS.
- Comparison between the available processor architectures.

The recorded 1024 × 1024 matrix multiplication results are presented in Section 5.

The original project proposal also included smaller, larger, and non-power-of-two matrix dimensions. Exact coverage of every requested dimension must be confirmed from the original CSV files before claiming that all dimensions were successfully tested.

4.4 Memory Bandwidth Experiments

Memory bandwidth experiments were conducted to understand how quickly each computing environment could move data during the benchmark.

Separate measurements were recorded for the CPU, GPU, and TPU environments. The collected results were saved as CSV files.

The combined memory bandwidth results were used to compare the recorded measurements and support the Roofline analysis.

4.5 Numerical Precision Experiments

The precision experiments examined FP32, FP16, and BF16 where supported by the tested environment.

The purpose was to compare execution time and numerical differences between precision formats.

FP32 was treated as the reference format for numerical error comparisons. Reduced-precision outputs were compared with the corresponding reference results.

INT8 was included in the broader architectural discussion, but no measured INT8 result is claimed in this paper.

4.6 Host-Device Transfer Experiments

The GPU transfer experiments measured data movement between CPU memory and GPU memory.

Both transfer directions were considered:

- CPU to GPU.
- GPU to CPU.

The tested transfer sizes included 1 MiB, 4 MiB, 16 MiB, 64 MiB, and 128 MiB.

Transfer bandwidth was calculated using the transferred data size and measured transfer time.

4.7 AI Workload Experiments

Several AI workloads were used to study performance beyond simple matrix multiplication.

The selected workloads included:

ResNet-50: A convolutional neural network commonly used for image recognition. The model contains approximately 25.56 million parameters.

DistilBERT: A smaller Transformer-based language model derived from BERT. The tested configuration used a sequence length of 128 and contained approximately 66.36 million parameters.

LSTM: A recurrent neural network workload with approximately 232,714 parameters. The recorded configuration used a sequence length of 100 and feature dimension of 64.

FashionMNIST CNN: A convolutional neural network trained on the FashionMNIST dataset. This experiment was used to compare CPU and GPU training performance.

CPU and GPU measurements were available for the selected inference workloads. A complete and verified TPU inference comparison was not available for every model.

4.8 CNN Training Experiment

A CNN model was trained using the FashionMNIST dataset on the available CPU and GPU environments.

The recorded training configuration was:

Parameter	Value
Dataset	FashionMNIST
Model	CNN
Model parameters	206,922
Batch size	128
Epochs	2
Learning rate	0.001
Devices	CPU and NVIDIA T4 GPU

The experiment recorded total training time, peak GPU memory usage where available, and test accuracy.

A TPU training result was not recorded, so the training comparison is limited to the CPU and GPU.

4.9 Performance Analysis

The collected results were analyzed using several computer architecture concepts.

The Roofline Model was used to examine the relationship between arithmetic intensity, memory bandwidth, and computational performance.

Amdahl's Law was used to understand theoretical parallel speedup.

The CPU profiler and thread scaling experiments were used to examine execution behavior and the effects of changing CPU thread counts.

4.10 Data Organization and Reproducibility

The project files were organized in Google Drive using the following directory structure:

```
AI_Inference_Precision_Comparison/
│
├── Results/
│   ├── Raw_CSV/
│   └── Graphs/
│
├── Code/
│
└── Documentation/
```

The CSV files preserve benchmark measurements, while the graphs provide visual comparisons of selected results.

This organization supports future review of the experiments and allows the collected results to be checked against the paper.

5. Experimental Results

This section presents the recorded experimental results. The numerical values are reported according to the available project measurements.

5.1 Matrix Multiplication Performance

The 1024 × 1024 matrix multiplication experiment showed clear differences between the tested computing environments.

Device	Execution Time	Performance
Intel Xeon CPU	16.136 ms	133.08 GFLOPS
NVIDIA Tesla T4 GPU	0.730 ms	2941.3 GFLOPS
Google Cloud TPU v5e-1	0.385 ms	5577.9 GFLOPS

The TPU recorded the highest performance in this particular experiment, followed by the NVIDIA T4 GPU. The CPU recorded the lowest throughput among the three tested environments.

These measurements show that specialized parallel processors can provide substantial performance improvements for suitable matrix multiplication workloads.

However, the results should be interpreted in the context of the tested software environment, matrix size, and measurement method. They do not establish that the TPU will outperform the GPU or CPU in every type of AI workload.

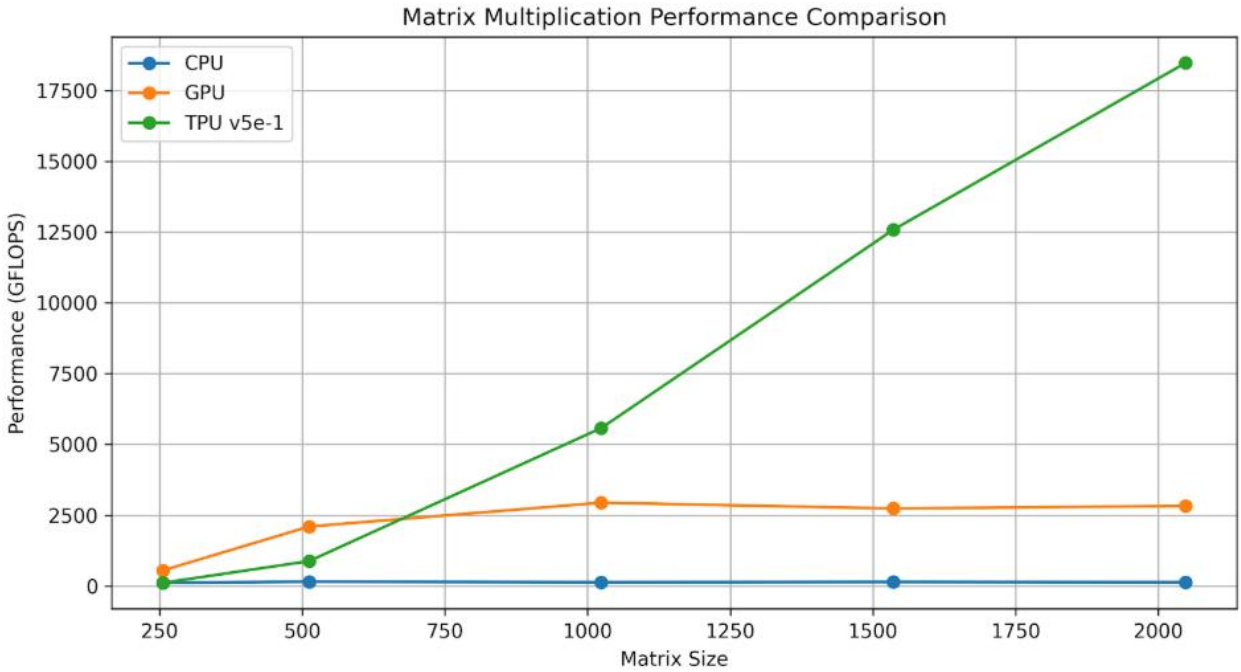


Figure 4: Matrix Multiplication Performance (GFLOPS) Across CPU, GPU, and TPU

5.2 Memory Bandwidth Results

The recorded memory bandwidth measurements at approximately 192 MB were:

Device	Recorded Bandwidth
Intel Xeon CPU	6.47 GB/s
NVIDIA Tesla T4 GPU	233.66 GB/s
Google Cloud TPU v5e-1	335.73 GB/s

The GPU and TPU recorded substantially higher bandwidth than the CPU in this experiment.

This difference helps explain why memory-intensive operations can behave differently across processor architectures.

The measured CPU value should be distinguished from the CPU Roofline bandwidth ceiling reported later in this paper. They represent different measurements and should not be treated as interchangeable.

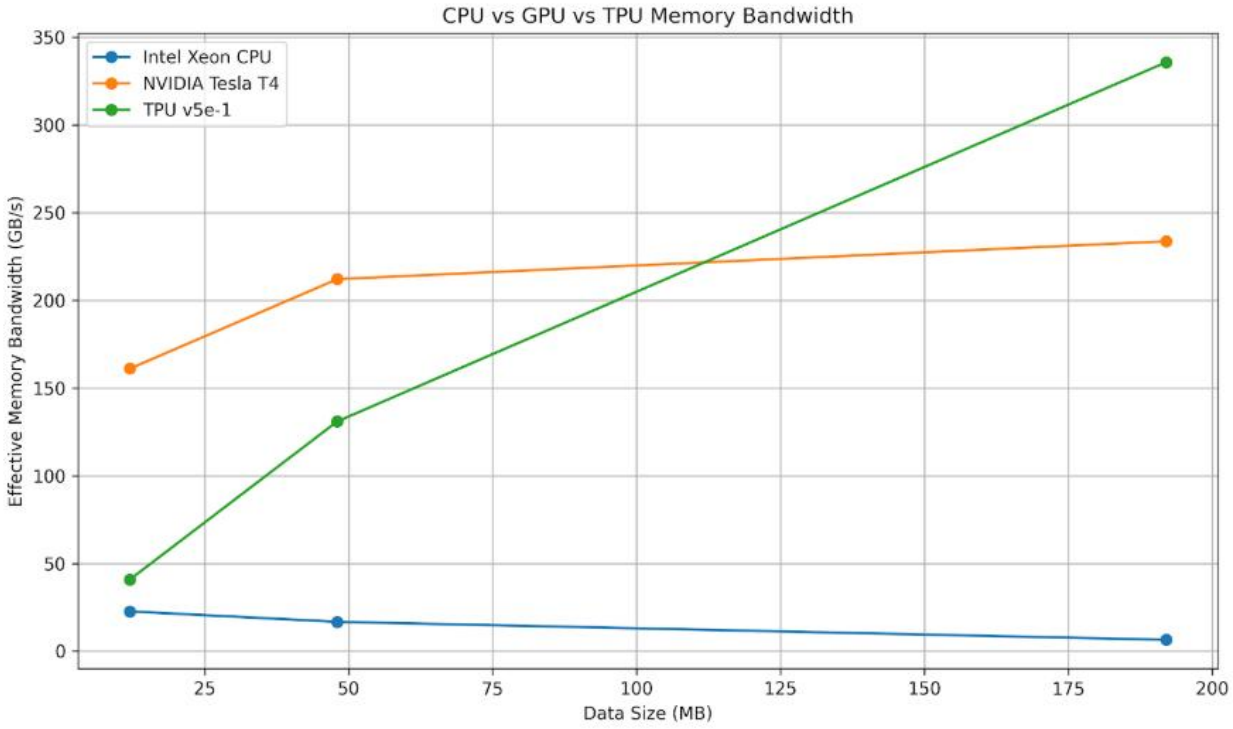


Figure 5: Matrix Multiplication Latency Comparison Across CPU, GPU, and TPU

5.3 Numerical Precision Results

The precision experiments showed that the effect of changing numerical formats depended on the processor architecture.

5.3.1 CPU Precision Results

The recorded CPU execution times for the 1024 matrix precision experiment were:

Precision	Execution Time
FP32	36.998 ms
FP16	3844.8 ms
BF16	3917.9 ms

In the tested CPU software environment, FP16 and BF16 execution times were considerably higher than FP32.

This result demonstrates that using a smaller numerical format does not automatically improve performance. Hardware support and software implementation can have a major effect on execution time.

These values belong to the precision benchmark and should not be confused with the separate CPU matrix multiplication result in Section 5.1.

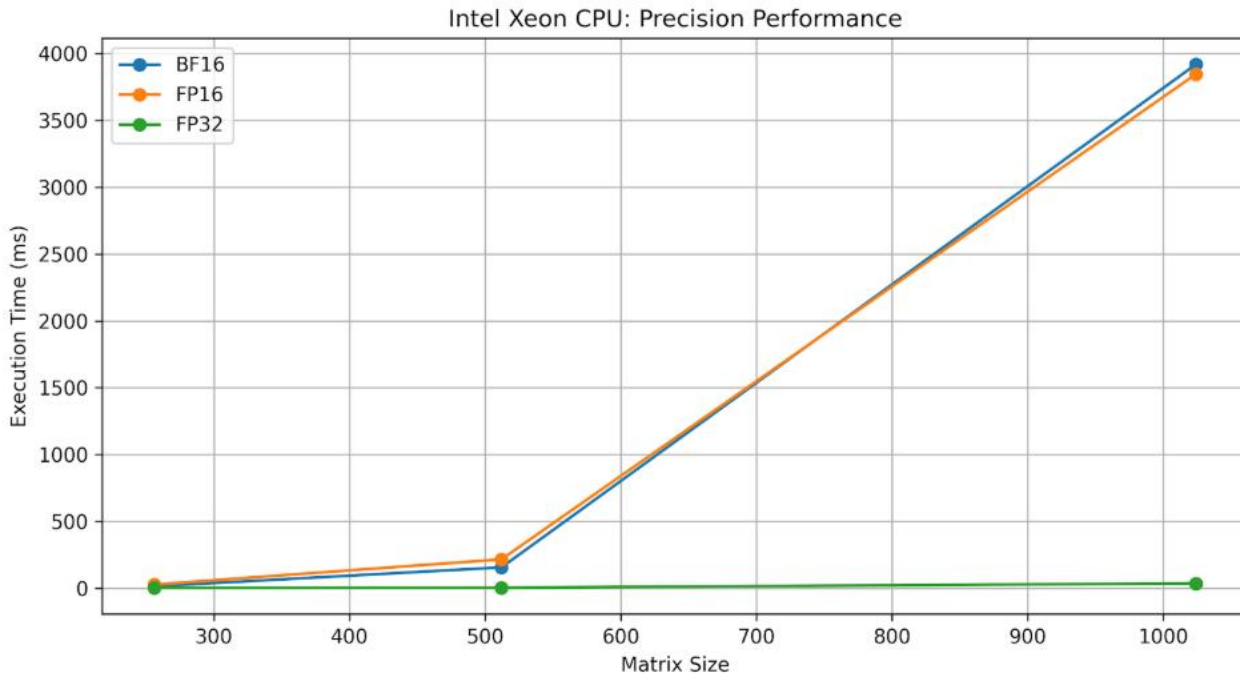


Figure 6: Numerical Precision Execution Time on Intel Xeon CPU

5.3.2 GPU Precision Results

The recorded NVIDIA T4 GPU execution times for the 1024 matrix experiment were:

Precision	Execution Time
FP32	0.854 ms
FP16	0.1666 ms
BF16	1.492 ms

FP16 achieved the lowest recorded execution time in this experiment.

The BF16 result was slower than FP32 in the recorded software environment. This indicates that the benefits of a numerical format depend on how effectively the device and framework support that format.

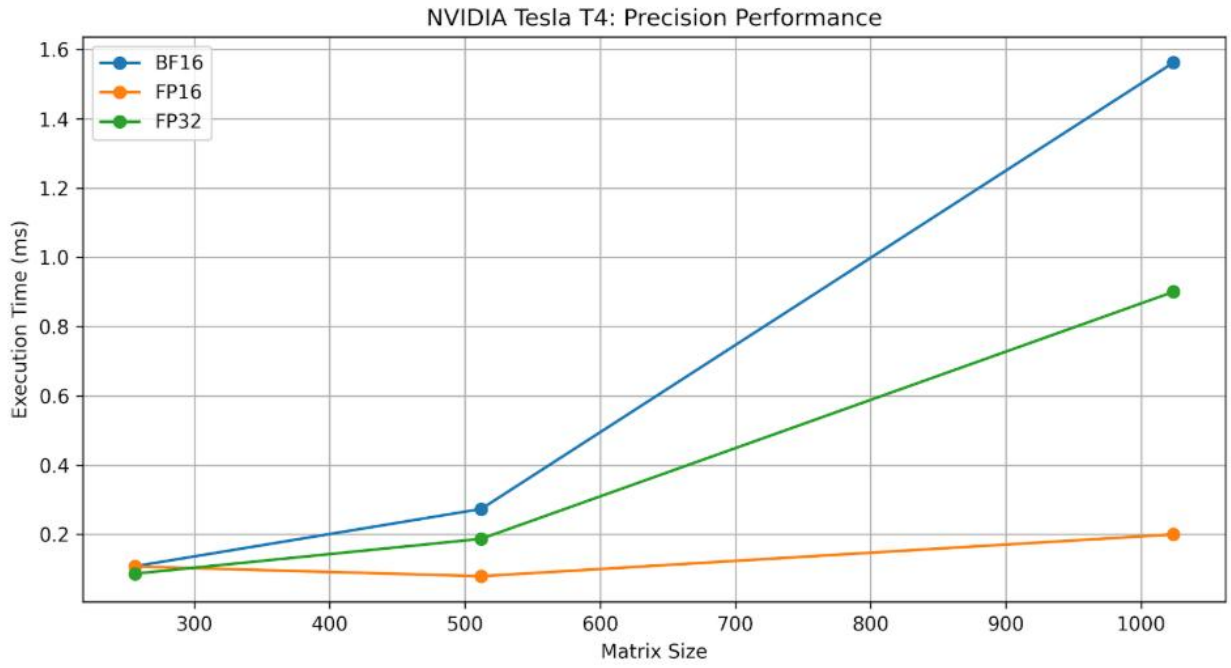


Figure 7: Numerical Precision Execution Time on NVIDIA Tesla T4 GPU

5.3.3 Numerical Error

The precision experiments also examined the differences between reduced-precision outputs and reference results.

The recorded TPU maximum absolute errors included:

Matrix Dimension	Maximum Absolute Error
256 × 256	2.29×10^{-5}
1024 × 1024	7.63×10^{-5}

For the T4 FP16 experiment at dimension 1024, the recorded error measurements were:

Error Measurement	Value
Maximum absolute error	0.079193
Mean absolute error	0.009036
Root mean square error	0.011491

These results show that numerical differences can occur when using reduced precision. The size of the error depends on the operation, numerical format, and implementation.

5.4 Host-Device Transfer Results

The GPU transfer experiment measured bandwidth in both directions.

Transfer Size	CPU to GPU	GPU to CPU
1 MiB	2.36 GB/s	2.81 GB/s
4 MiB	1.40 GB/s	4.04 GB/s
16 MiB	2.20 GB/s	4.54 GB/s
64 MiB	4.34 GB/s	1.42 GB/s
128 MiB	4.30 GB/s	1.41 GB/s

The recorded results show that transfer bandwidth varied with data size and transfer direction.

This variation is important because AI workloads often require data to be transferred before computation begins. As a result, the total time required by an application may be greater than the time spent performing calculations on the GPU.

These results apply to the tested GPU transfer environment and should not be treated as TPU transfer measurements.

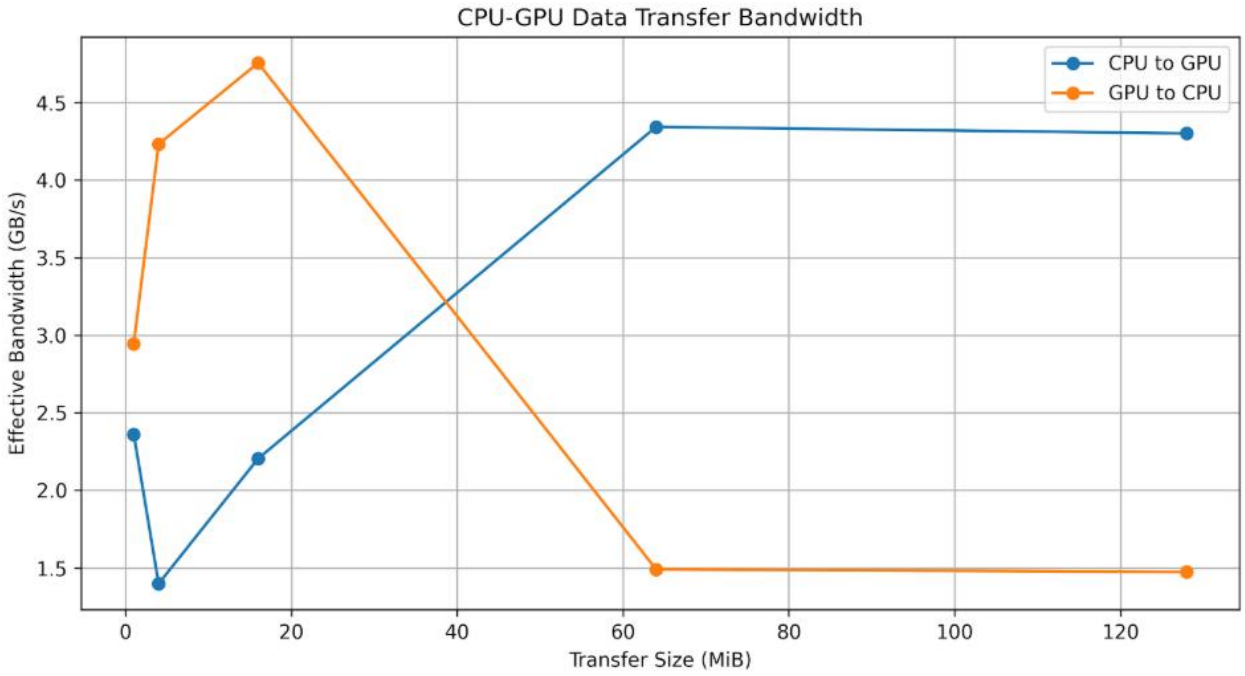


Figure 8: Host-to-Device and Device-to-Host Transfer Bandwidth

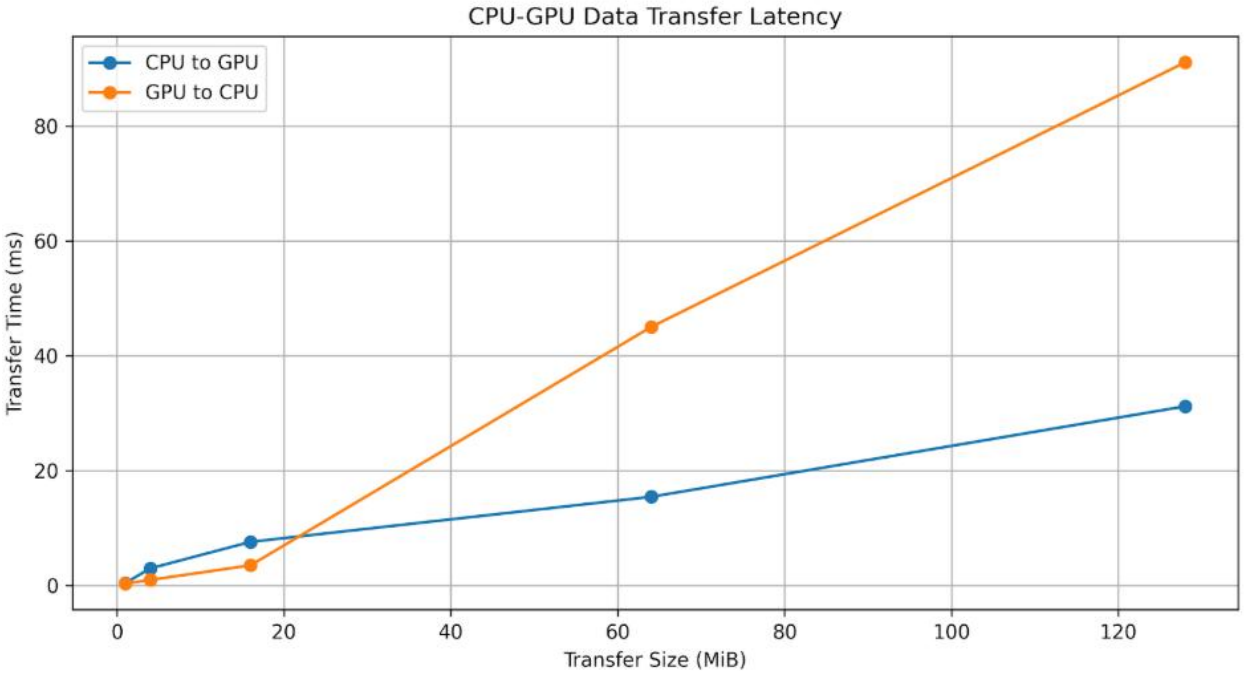


Figure 9: Host-to-Device and Device-to-Host Transfer Latency

5.5 ResNet-50 Inference

ResNet-50 was included to examine the performance of a convolutional neural network workload.

The model contained approximately 25,557,032 parameters. The recorded experiments considered batch sizes from 1 to 32.

CPU and GPU inference measurements were collected and used to generate latency, throughput, and speedup graphs.

The results demonstrate the importance of batch size when evaluating inference performance. Larger batches can allow a parallel processor to process more samples together, but they may also increase memory requirements and affect latency.

The exact numerical values for each batch size should be taken from the original ResNet-50 CSV files and corresponding graphs.

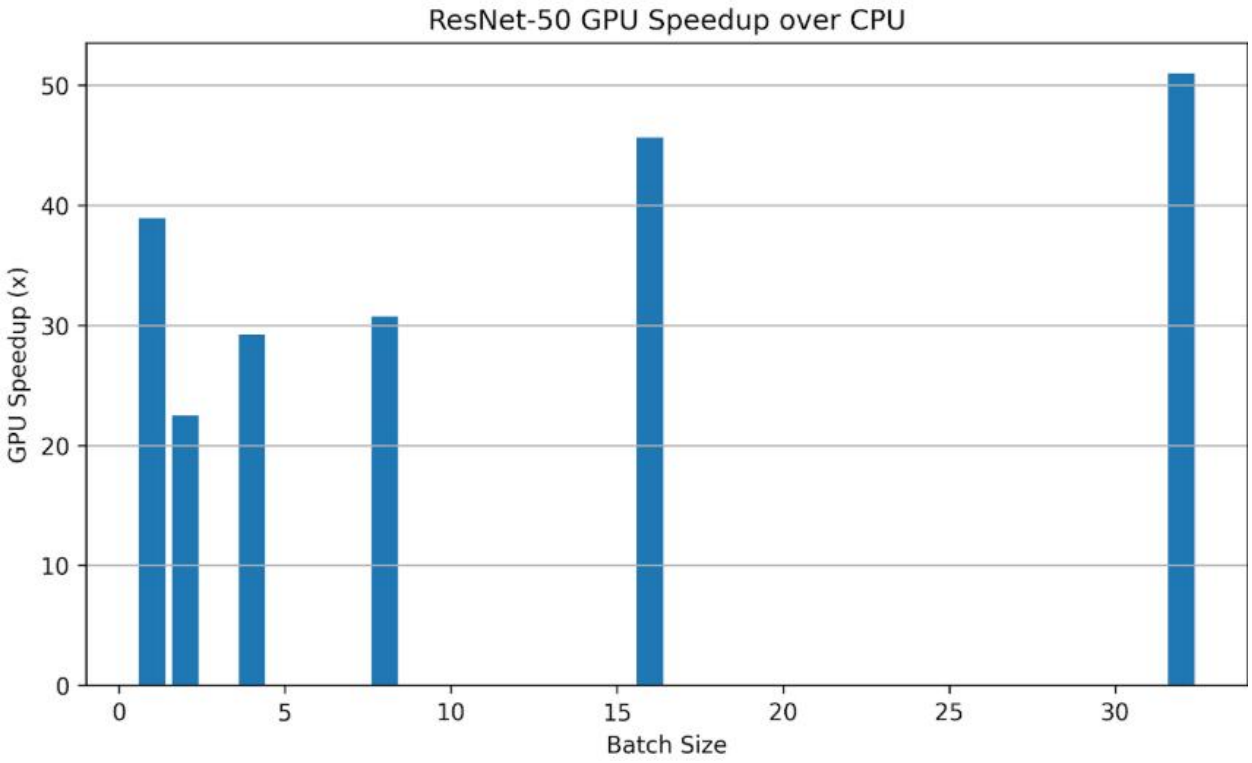


Figure 10: ResNet-50 GPU Speedup Over CPU Across Different Batch Sizes

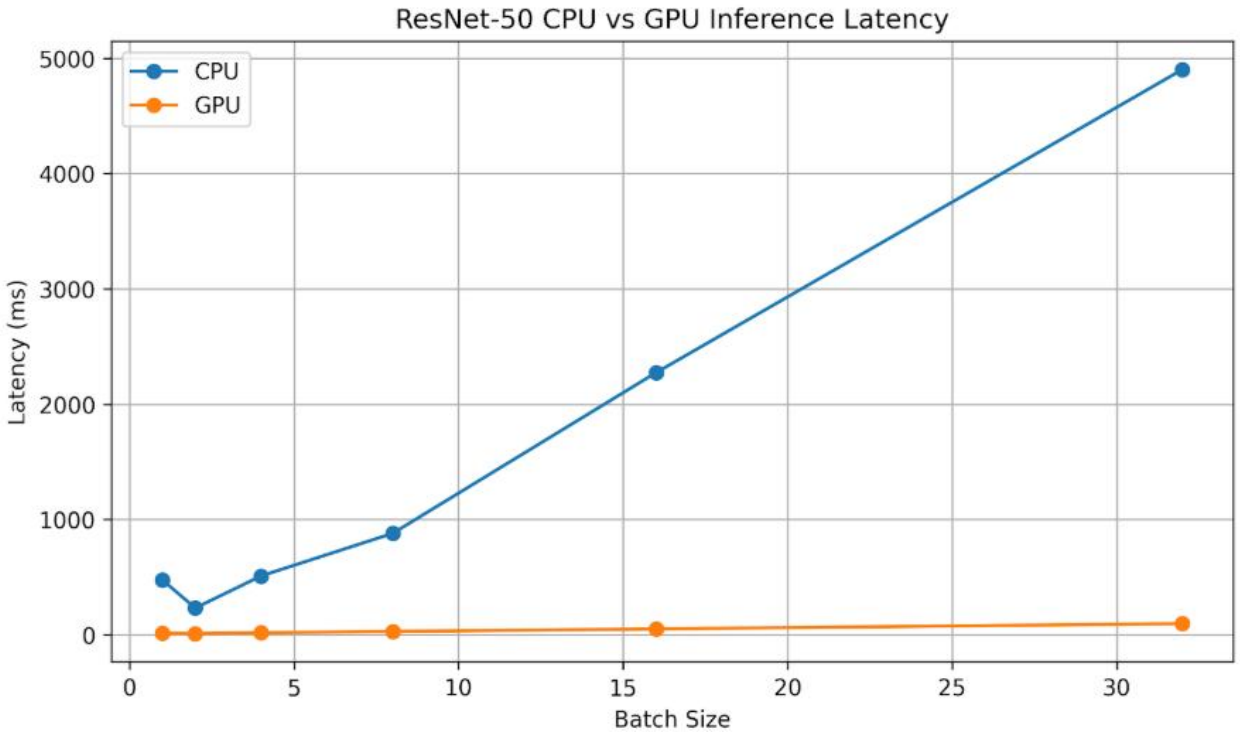


Figure 11: ResNet-50 Inference Latency Across Different Batch Sizes

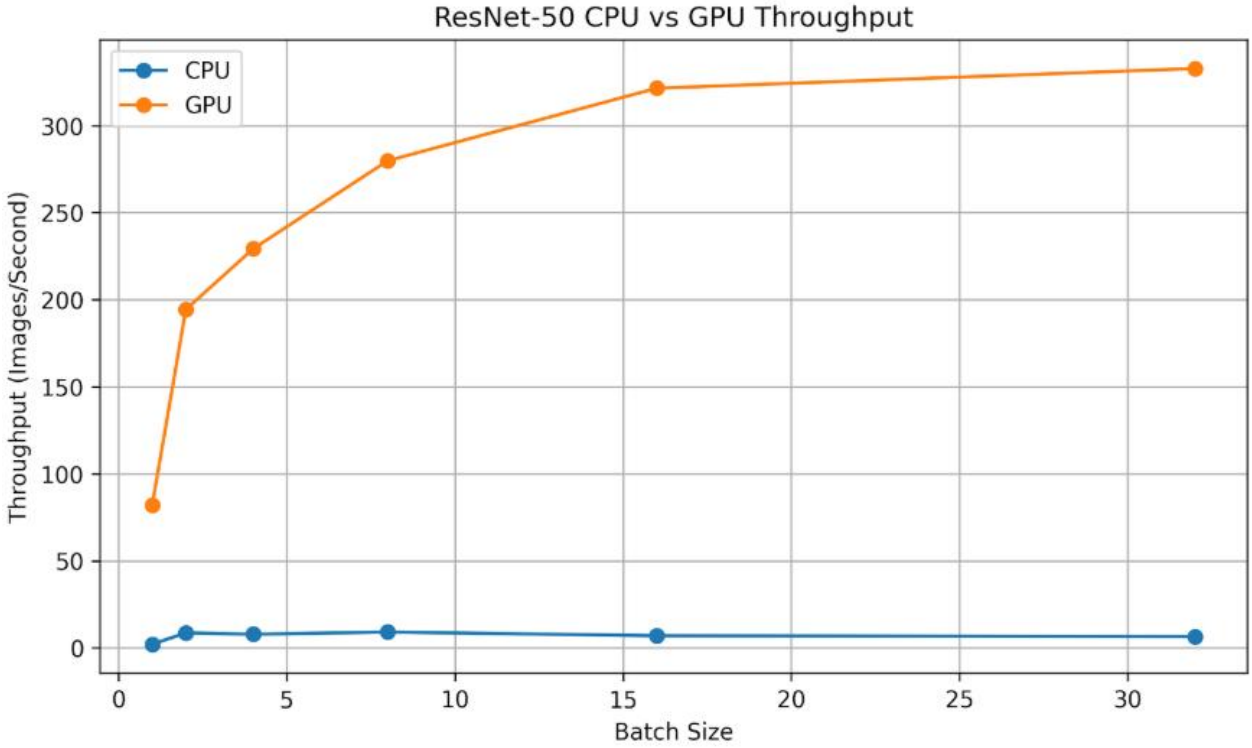


Figure 12: ResNet-50 Inference Throughput Across Different Batch Sizes

5.6 DistilBERT Inference

DistilBERT was used to study a Transformer-based language processing workload.

The recorded model configuration contained approximately 66,362,880 parameters and used a sequence length of 128.

CPU and GPU inference measurements were collected.

Transformer models involve operations such as matrix multiplication and attention calculations. Their execution performance can therefore be influenced by processor architecture, memory access, and numerical precision.

The recorded experiments provide a comparison between the available CPU and GPU environments. A complete TPU comparison is not claimed.

5.7 LSTM Inference

An LSTM workload was also included to examine recurrent neural network computation.

The recorded configuration contained approximately 232,714 parameters and used a sequence length of 100 with a feature dimension of 64.

CPU and GPU inference measurements were saved in the project's raw CSV directory.

Unlike a simple matrix multiplication benchmark, recurrent workloads involve sequential dependencies between processing steps. This can influence how effectively parallel hardware is used.

The LSTM experiment therefore provides another perspective on how workload structure affects processor performance.

5.8 FashionMNIST CNN Training Results

The FashionMNIST CNN training experiment compared the CPU and NVIDIA T4 GPU.

The recorded results were:

Measurement	CPU	NVIDIA T4 GPU
Total training time	100.371 s	21.586 s
Test accuracy	86.97%	87.41%

Peak GPU memory Not recorded as a comparable CPU value 478 MB

The measured speedup was approximately:

$$Speedup = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}$$

$$Speedup = \frac{100.371}{21.586}$$

$$Speedup \approx 4.65$$

The GPU completed the recorded training experiment in less time than the CPU.

The test accuracy values were also close, with a small difference between the two recorded results.

The experiment demonstrates that GPU acceleration can improve training time for suitable neural network workloads. However, the result is specific to the selected model, dataset, training configuration, and runtime environment.

6. Computer Architecture Analysis

The experimental measurements can be interpreted using computer architecture concepts. This section discusses the Roofline Model, Amdahl's Law, CPU profiling, and thread scaling.

6.1 Roofline Model Analysis

The Roofline Model was used to examine the relationship between computational performance and memory bandwidth.

Separate Roofline graphs were generated for the three tested environments:

1. Intel Xeon CPU.
2. NVIDIA Tesla T4 GPU.
3. Google Cloud TPU v5e-1.

The recorded bandwidth ceilings used in the corrected Roofline input data were:

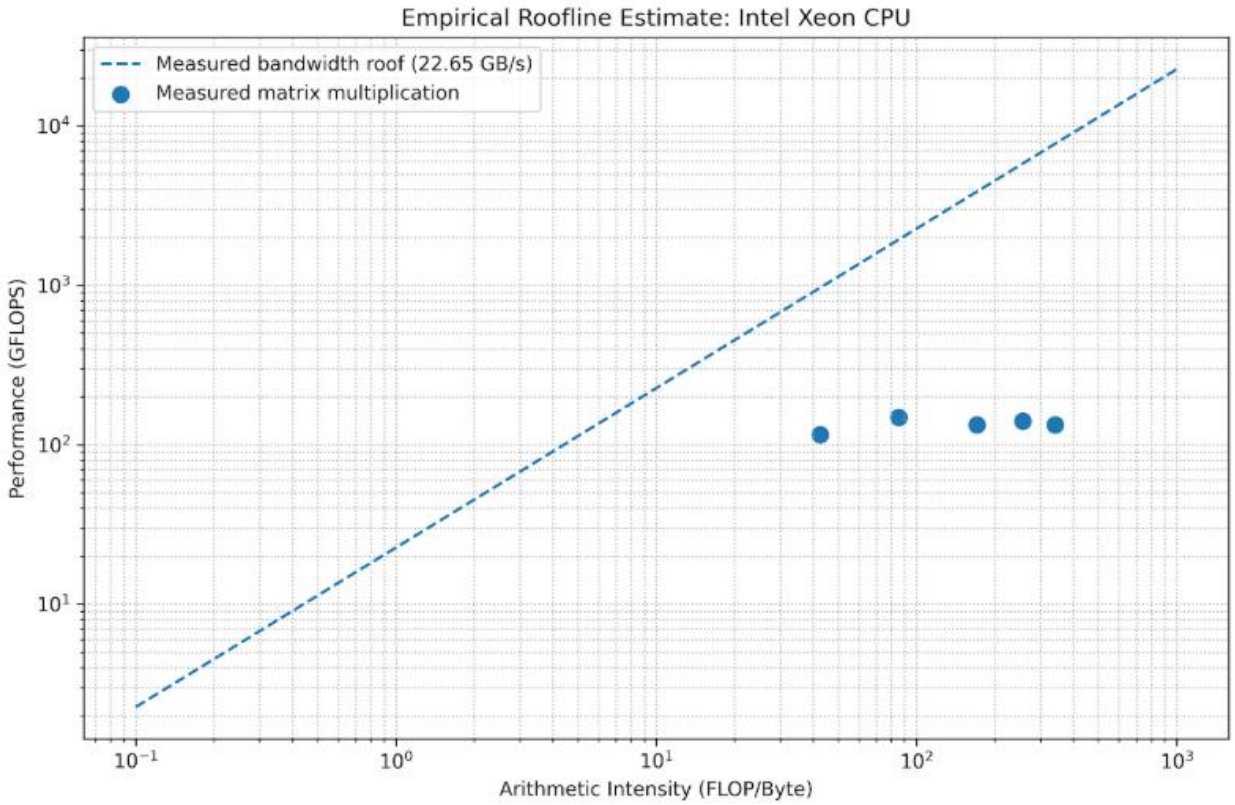
Device	Roofline Bandwidth Ceiling

Intel Xeon CPU	22.648432 GB/s
NVIDIA Tesla T4 GPU	233.660000 GB/s
Google Cloud TPU v5e-1	335.734770 GB/s

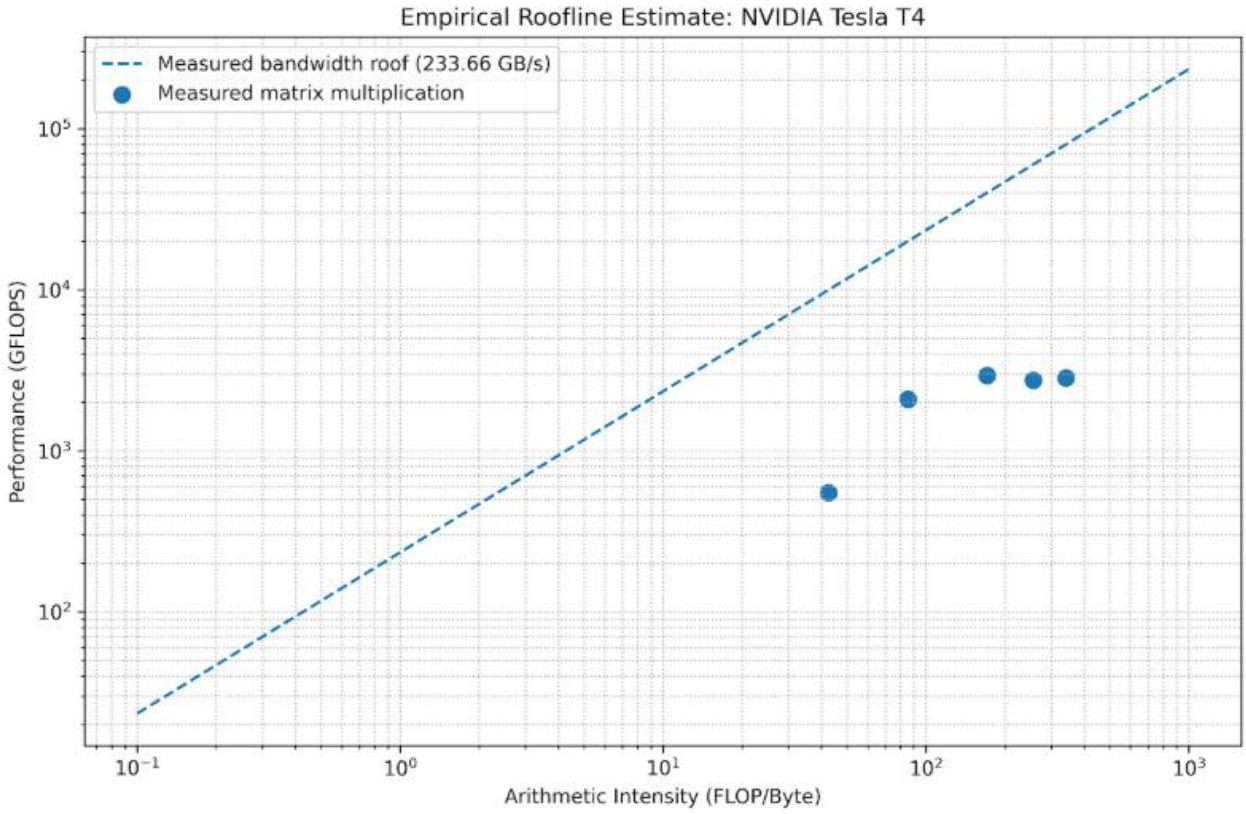
These values are Roofline input ceilings and should not be confused with every individual measured bandwidth result.

The generated graphs provide a visual representation of the estimated performance limits of each device.

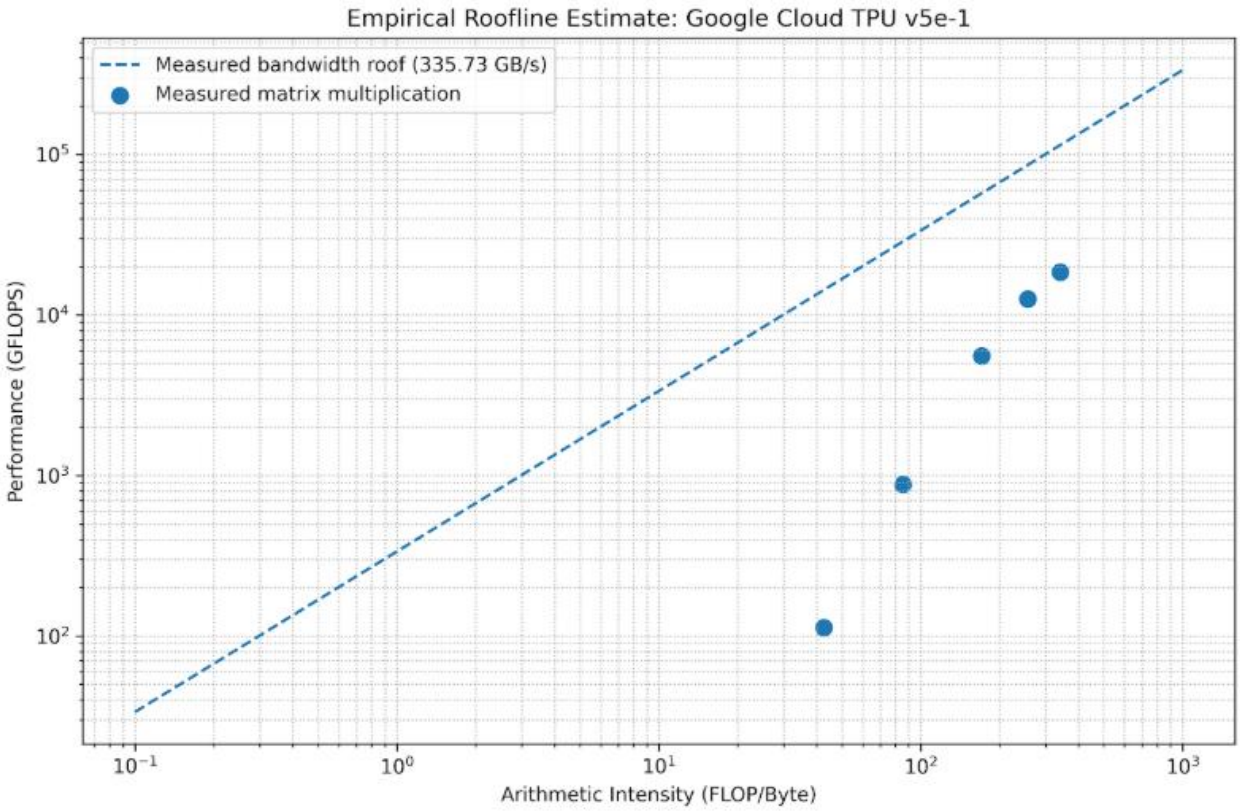
The analysis helps explain that computational performance depends on both processing capability and data availability. A workload may fail to reach a device's computational limit when its memory requirements become the main performance restriction.



- Figure 13: Intel Xeon CPU Roofline Model — `intel_xeon_cpu_roofline.png`



- Figure 14: NVIDIA Tesla T4 Roofline Model — `nvidia_tesla_t4_roofline.png`



- Figure 15: Google Cloud TPU v5e-1 Roofline Model — `google_cloud_tpu_v5e_1_roofline.png`

6.2 Amdahl's Law Analysis

Amdahl's Law was applied to understand the relationship between parallel execution and theoretical speedup.

One recorded example used a parallel fraction of 0.50, a serial fraction of 0.50, and two processing units.

The theoretical speedup was:

$$S = \frac{1}{(1 - 0.50) + \frac{0.50}{2}}$$

$$S = \frac{1}{0.50 + 0.25}$$

$$S = \frac{1}{0.75} \approx 1.333$$

This result shows that when half of a program remains serial, adding processing units cannot provide unlimited speedup.

The project also compared the theoretical Amdahl's Law results with the measured CNN training speedup of approximately 4.65×.

These values represent different analyses. The measured CNN speedup comes from an actual CPU-GPU experiment, while the Amdahl's Law value is a theoretical result based on an assumed parallel fraction and processing-unit count.

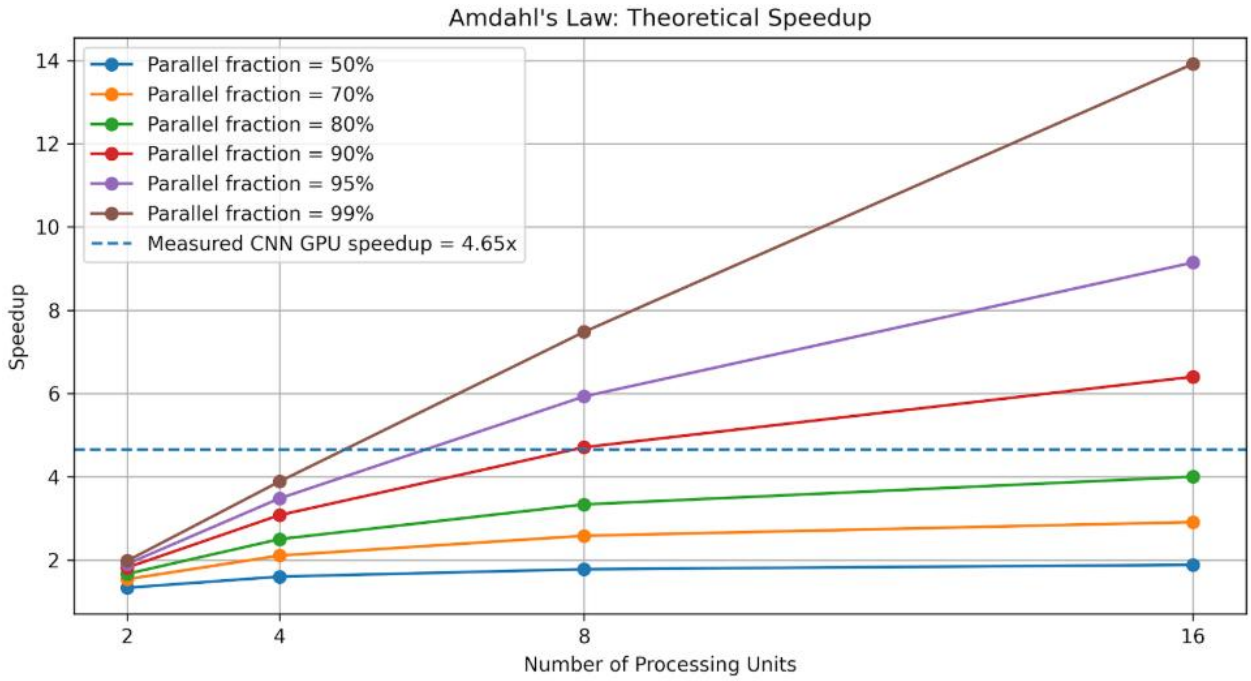


Figure 16: Theoretical Speedup According to Amdahl's Law.

6.3 CPU Performance Analysis

The project included an attempt to examine CPU execution behavior using profiling tools.

The Linux `perf` utility was not available in the Google Colab environment. Therefore, hardware performance counters such as instruction count, cycles, and CPI could not be collected directly.

Instead, the PyTorch profiler was used to examine a CPU matrix multiplication workload.

For the recorded 512 × 512 CPU matrix multiplication experiment, the profiler reported a total time of approximately 106,488 microseconds across five calls.

This provides information about the observed execution time in the profiling environment. However, it does not provide a complete Iron Law breakdown because instruction count and hardware cycle information were unavailable.

The limitation is important because profiler execution time alone cannot be used to calculate an exact CPI value.

6.4 CPU Thread Scaling

An additional experiment examined CPU performance under different thread configurations.

The purpose was to understand how changing the number of CPU threads influences execution time.

The experiment generated a CSV file and a corresponding graph. The results can be used to study the relationship between parallel execution and available CPU resources.

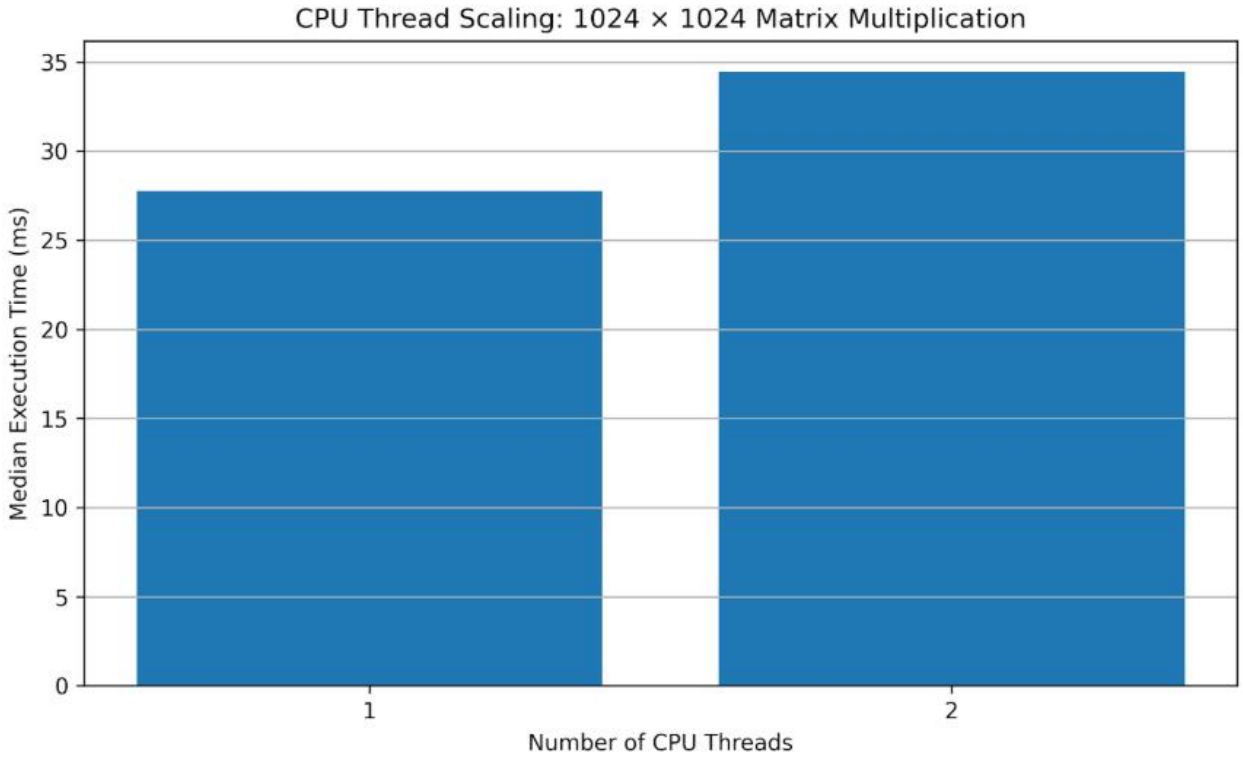


Figure 17: CPU Performance Under Different Thread Configurations.

Thread scaling does not always produce a proportional performance improvement. The final result may be influenced by scheduling overhead, memory access, synchronization, and the number of physical processing resources available.

The thread scaling results are included as an additional profiling experiment.

7. Discussion

The experimental results demonstrate that numerical precision and processor architecture have a strong relationship with AI performance.

The matrix multiplication experiment showed that the tested GPU and TPU environments achieved considerably higher throughput than the CPU for the recorded 1024 × 1024 workload. This is consistent with the parallel processing design of specialized accelerators.

The precision experiments produced an important observation: reduced precision did not improve performance on every tested architecture. FP16 provided a substantial speed improvement on the NVIDIA T4 GPU, but the CPU FP16 and BF16 experiments were much slower than FP32 in the recorded software environment.

This indicates that numerical precision should not be selected only according to the number of bits used to represent a value. Hardware support and framework implementation must also be considered.

Memory bandwidth was another important factor. The recorded GPU and TPU memory bandwidth measurements were higher than the CPU result. This can help explain the performance differences in workloads that require frequent data movement.

The host-device transfer experiment also demonstrated that transferring data introduces additional overhead. Therefore, evaluating only the time spent performing calculations may not provide a complete picture of application performance.

The CNN training experiment showed that the GPU completed the recorded training task approximately 4.65 times faster than the CPU. The accuracy values were close, indicating that the measured performance difference did not correspond to a large difference in test accuracy for this experiment.

The results also show why different AI workloads should be evaluated separately. Matrix multiplication, convolutional networks, Transformer models, and recurrent networks have different computational and memory-access characteristics.

Overall, the experiments suggest that the best processor and precision format depend on the workload, hardware capabilities, software support, and required numerical accuracy.

8. Answers to the Research Questions

8.1 How does numerical precision affect AI performance?

Numerical precision affects both computational requirements and numerical accuracy.

Lower-precision formats can reduce storage requirements and improve computational performance when the hardware supports efficient operations in those formats.

However, the CPU and GPU precision experiments showed that the performance effect was not identical across devices. FP16 performed well on the tested T4 GPU, while FP16 and BF16 were slower than FP32 in the tested CPU environment.

8.2 Which tested architecture provided better matrix multiplication performance?

For the recorded 1024 × 1024 matrix multiplication experiment, the Google Cloud TPU v5e-1 achieved the highest measured throughput at approximately 5577.9 GFLOPS.

The NVIDIA T4 GPU achieved approximately 2941.3 GFLOPS, while the Intel Xeon CPU achieved approximately 133.08 GFLOPS.

These values describe the specific benchmark conditions rather than the overall performance of every device in all applications.

8.3 Does reduced precision always improve performance?

No. The experimental results show that reduced precision does not always provide a performance improvement.

FP16 achieved a lower execution time than FP32 in the tested T4 GPU experiment. However, the CPU precision experiment recorded much higher execution times for FP16 and BF16 than for FP32.

The result depends on hardware support, software implementation, and the type of computation.

8.4 How do memory bandwidth and data transfer affect performance?

Memory bandwidth determines how quickly data can be supplied to a processor. A workload may become memory-bound when the processor cannot receive data quickly enough.

Host-device transfers also introduce additional time. Therefore, even a fast GPU may experience reduced end-to-end performance when a workload requires frequent data transfers.

8.5 What is the relationship between numerical precision and accuracy?

Lower-precision formats can introduce rounding differences compared with FP32 calculations.

The recorded error measurements show that reduced-precision outputs can differ from the reference output. The amount of error depends on the numerical format, workload, and implementation.

Therefore, performance improvements should be considered alongside numerical accuracy.

8.6 How do Roofline Model and Amdahl's Law explain performance?

The Roofline Model explains whether a workload is more likely to be limited by memory bandwidth or computational capability.

Amdahl's Law explains how the serial portion of a program limits the speedup achievable through parallel processing.

Together, these models help explain why additional computational resources do not always produce proportional performance improvements.

9. Original Research Question and Hypothesis

9.1 Original Research Question

How does numerical precision influence computational performance and numerical accuracy across different processor architectures, and to what extent do memory bandwidth, parallel processing, and hardware support affect the benefits of reduced-precision AI computation?

9.2 Research Hypothesis

The initial hypothesis was that lower-precision formats would generally improve AI computational performance by reducing data requirements and enabling more efficient hardware execution.

However, the expected improvement was considered dependent on processor architecture and software support.

The experimental findings supported this conditional hypothesis. FP16 performed well on the tested NVIDIA T4 GPU, but reduced precision did not provide the same benefit in the tested CPU environment.

The findings therefore indicate that numerical precision and hardware architecture must be considered together when evaluating AI performance.

10. Limitations of the Study

Although the experiments provide useful comparisons, several limitations should be considered when interpreting the results.

Limited hardware availability: The experiments were conducted using the computing resources available through Google Colab. These environments may differ from dedicated local or data-center systems.

CPU identification: The runtime reported an Intel Xeon CPU at 2.00 GHz, but the exact processor model was not established.

NPU availability: No physical NPU was available for direct testing. NPU architecture is therefore discussed conceptually rather than through measured benchmark results.

TPU workload coverage: TPU measurements were collected for selected experiments. A complete TPU comparison for every AI workload and training experiment was not available.

INT8 measurements: INT8 was discussed as an AI precision format, but a verified INT8 benchmark was not recorded.

Hardware profiling: The `perf` utility was unavailable in the Colab environment. Therefore, instruction count, CPU cycles, and CPI hardware counters could not be obtained.

Runtime restrictions: TPU usage limitations affected the ability to conduct further experiments.

Benchmark variability: Cloud computing environments may experience performance variation because of shared resources, runtime conditions, and software configurations.

Statistical reporting: The original benchmark files should be used to verify the number of repetitions, warm-up procedures, mean, median, standard deviation, and tail-latency measurements before these are reported as completed requirements.

These limitations do not invalidate the recorded results, but they define the conditions under which the findings should be interpreted.

11. Conclusion

This research investigated the impact of numerical precision on AI performance across different processor architectures. The study compared an Intel Xeon CPU, an NVIDIA Tesla T4 GPU, and a Google Cloud TPU v5e environment using selected computational and AI workloads.

The matrix multiplication experiment showed substantial performance differences between the tested devices. At a matrix dimension of 1024 × 1024, the TPU achieved approximately 5577.9

GFLOPS, the GPU achieved approximately 2941.3 GFLOPS, and the CPU achieved approximately 133.08 GFLOPS.

The precision experiments demonstrated that lower numerical precision does not automatically provide better performance. FP16 achieved the lowest recorded execution time in the T4 GPU precision experiment, whereas FP16 and BF16 were considerably slower than FP32 in the tested CPU environment.

Memory bandwidth and data transfer were also important factors. The experiments showed that processing speed alone does not fully describe the performance of an AI application. Data movement and memory access can influence the overall execution time.

The FashionMNIST training experiment further demonstrated the benefit of GPU acceleration for the selected workload. The NVIDIA T4 completed the recorded training experiment in approximately 21.586 seconds, compared with 100.371 seconds on the CPU, resulting in a measured speedup of approximately 4.65×.

The Roofline Model, Amdahl's Law, and CPU profiling provided additional architectural explanations for the observed results.

In conclusion, the research indicates that selecting an appropriate numerical precision format requires consideration of the processor architecture, hardware support, software implementation, workload characteristics, and required accuracy. There is no single numerical format or processor architecture that is automatically the best choice for every AI application.

12. Future Work

Future work can extend this research in several directions.

First, the experiments can be repeated on additional hardware platforms, including a physical NPU, to provide a more complete architectural comparison.

Second, INT8 quantization experiments can be performed to examine the relationship between model size, inference speed, and accuracy.

Third, additional TPU experiments can be conducted to compare CNN, Transformer, and recurrent workloads under more consistent conditions.

Fourth, the benchmark methodology can be extended with more repetitions, detailed statistical reporting, and batch-1 p95 and p99 latency measurements.

Finally, hardware performance counters can be collected on a suitable local system to provide a more complete CPU Iron Law analysis.

These improvements would help provide a broader understanding of how processor architecture and numerical precision influence modern AI applications.

13. References

The following references provide background for the architecture concepts, numerical formats, performance models, and AI models discussed in this research. Bibliographic details and citation formatting should be checked against the final sources used before submission.

[1] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.

[2] NVIDIA, *NVIDIA T4 Tensor Core GPU: Product Brief*. NVIDIA Corporation.

[3] Google Cloud, *Cloud TPU Documentation*. Google Cloud Documentation.

[4] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[5] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in *Proceedings of the AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[6] PyTorch, *PyTorch Documentation*. Available: https://pytorch.org/docs/

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter," arXiv:1910.01108, 2019.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

14. Appendices

Appendix A: Project Directory Structure

The experimental files were organized in Google Drive as follows:

```
AI_Inference_Precision_Comparison/
│
├── Results/
│   ├── Raw_CSV/
│   │
│   └── Graphs/
│
├── Code/
│
└── Documentation/
```

The Raw_CSV directory contains the recorded experimental data, while the Graphs directory contains visual representations of selected results.

Appendix B: Important Experimental Graphs

The following graphs should be inserted into the relevant sections of the paper.

Figure	Graph File	Recommended Section
1	matrix_performance_gflops.png	Section 5.1
2	matrix_latency_comparison.png	Section 5.1
3	intel_xeon_cpu_precision_time.png	Section 5.3
4	nvidia_tesla_t4_precision_time.png	Section 5.3
5	host_device_transfer_latency.png	Section 5.4
6	host_device_transfer_bandwidth.png	Section 5.4
7	resnet50_latency_comparison.png	Section 5.5
8	ResNet-50 throughput graph	Section 5.5
9	ResNet-50 speedup graph	Section 5.5
10	amdahl_law_speedup.png	Section 6.2
11	intel_xeon_cpu_roofline.png	Section 6.1
12	nvidia_tesla_t4_roofline.png	Section 6.1
13	google_cloud_tpu_v5e_1_roofline.png	Section 6.1

Use the exact graph filenames from the Google Drive project folder. If a graph is missing, do not insert a substitute that represents a different experiment.

Appendix C: Important CSV Files

The following files were recorded in the project's Raw_CSV directory:

- `combined_matrix_benchmark.csv`
- `matrix_multiplication_cpu_gpu.csv`
- `matrix_multiplication_tpu_final.csv`
- `matrix_multiplication_tpu_validated.csv`
- `memory_bandwidth_cpu.csv`
- `memory_bandwidth_gpu.csv`
- `memory_bandwidth_tpu.csv`
- `combined_memory_bandwidth.csv`
- `combined_precision_all_sizes.csv`
- `roofline_input_data_corrected.csv`
- `lstm_gpu_inference.csv`